

Backend File Documentation

This section documents every Python file in the backend, including its purpose, key classes, and all public functions with their parameters and descriptions.

Entry Point & Application Bootstrap

[backend/app/main.py](#)

FastAPI application factory. Configures structured logging, registers ASGI middleware (SessionContext → SecurityHeaders → RateLimit → CORS), mounts all routers, and runs startup tasks via the lifespan context manager.

Method / Function	Parameters	Description
<code>_configure_logging()</code>	→ None	Sets up structured logging with a dedicated mailmind.audit logger. Attaches a FileHandler when AUDIT_LOG_FILE env var is set for an isolated audit sink.
<code>lifespan(app)</code>	app: FastAPI → AsyncGenerator	Startup: creates EmailQueue, calls init_db(), ensures ChromaDB directory, pre-warms Presidio/spaCy (eliminates 4s cold-start), checks LangSmith status.
<code>app = FastAPI(...)</code>	—	Creates FastAPI instance with title, description, version 2.0.0, and lifespan handler.
<code>init_observability(app)</code>	app: FastAPI → None	Registers global exception handlers and optional Sentry integration (from observability.py).
<code>app.add_middleware(...)</code>	—	Middleware stack (reverse order): RateLimit → SecurityHeaders → SessionContext → CORS.
<code>app.include_router(...)</code>	—	Mounts: main router, agent, demo, monitoring, feedback, waitlist, pii, compliance, sync routers.

Configuration

[backend/app/config/settings.py](#)

Single Settings class that reads all environment variables. Loaded once at import time; dotenv first, then Azure Key Vault takes precedence.

Method / Function	Parameters	Description
<code>_bool_env(name, default)</code>	name: str, default: bool → bool	Parses boolean env vars — accepts "1", "true", "yes".
<code>Settings.bootstrap_allowed_set</code>	→ set[str]	Property — lowercase set of always-allowed owner emails (never locked out by waitlist).
<code>Settings.is_production</code>	→ bool	Property — True when APP_ENV == "production".
<code>Settings.persistence_enabled</code>	→ bool	Property — True when DATABASE_URL is non-empty.
<code>Settings.azure_openai_base_endpoint</code>	→ str	Property — strips deployment path from Azure URL to get scheme+host only.

[backend/app/config/keyvault.py](#)

Azure Key Vault integration. Loads secrets into os.environ at startup so downstream code reads them like normal env vars.

Method / Function	Parameters	Description
<code>load_keyvault_into_env()</code>	→ None	Reads AZURE_KEYVAULT_URL; if set, fetches all secrets and writes to os.environ. Requires Azure Managed Identity or AZURE_CLIENT_* credentials.

ASGI Middleware

[backend/app/middleware.py](#)

Three custom ASGI middleware classes stacked on the FastAPI application.

Method / Function	Parameters	Description
<code>SecurityHeadersMiddleware.__call__</code>	<code>request</code> , <code>call_next</code>	Injects: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, and HSTS (when <code>HSTS_ENABLED=true</code> and HTTPS).
<code>RateLimitMiddleware.__call__</code>	<code>request</code> , <code>call_next</code>	Fixed-window per-IP rate limiter (default 100 req/min). Exempts <code>/api/health</code> , <code>/api/ready</code> , and <code>OPTIONS</code> . Returns HTTP 429 with <code>Retry-After</code> header.
<code>SessionContextMiddleware.__call__</code>	<code>request</code> , <code>call_next</code>	Binds a <code>ContextVar</code> (<code>current_session</code>) before the request and clears it after. Downstream code calls <code>get_current_session()</code> instead of passing sessions.

Database Base

[backend/app/db/base.py](#)

SQLAlchemy engine factory and session maker. Configures pool parameters from settings.

Method / Function	Parameters	Description
<code>create_engine(...)</code>	—	Creates async-compatible SQLAlchemy engine with <code>pool_size</code> , <code>max_overflow</code> , <code>pool_timeout</code> , <code>pool_pre_ping</code> , <code>pool_recycle</code> from Settings.
<code>SessionLocal</code>	—	sessionmaker factory; use as context manager or via FastAPI dependency.
<code>init_db()</code>	→ None	Calls <code>Base.metadata.create_all()</code> — creates all ORM tables on first run. No-op when <code>DATABASE_URL</code> is empty.

ORM Models

[backend/app/db/models.py](#)

All 15 SQLAlchemy ORM classes (tables). UUID primary keys throughout. Defines relationships, unique constraints, and JSON columns.

Class (Table)	Description
<code>User</code>	Core identity. UUID PK. One user can have multiple <code>OAuthAccounts</code> .
<code>OAuthAccount</code>	Connected email account (Gmail/Outlook). Fernet-encrypted tokens. <code>UniqueConstraint(provider, provider_account_id)</code> . Links to <code>User</code> .
<code>UserSession</code>	Short-lived session (24h). <code>token_hash = SHA-256(secret + raw_token)</code> . Links to <code>User</code> .
<code>Device</code>	Trusted browser fingerprint for Quick Login. Links to <code>User</code> .
<code>QuickLoginToken</code>	7-day auto-resume token. <code>status: ACTIVE/LOGGED_OUT/REVOKED/EXPIRED</code> .
<code>EmailEnrichment</code>	Triage + enrichment results. JSON columns: <code>axes</code> , <code>dynamic_weights</code> , <code>commitments</code> , <code>precedents</code> . <code>status: pending/done/error</code> .
<code>AuditLog</code>	Append-only compliance trail. No raw PII. <code>action</code> , <code>actor</code> , <code>details</code> (JSON).
<code>ToneProfile</code>	Stylometric profile per account. <code>profile</code> (JSON 8 features). One row per account.
<code>Feedback</code>	Product feedback. <code>rating</code> , <code>category</code> , <code>message</code> , <code>role</code> .

TriagePriorityOverride	User priority feedback loop. original vs override priority per email.
ProcessingMetric	SLA latency per stage. duration_ms, success, sla_met.
MailboxMessage	Server-side envelope mirror. state: active/done/deleted. sender, subject, snippet, received_at, is_read, is_starred.
MailboxSyncState	Delta cursor per account+folder. delta_cursor, backfill_done, last_synced_at, message_count.
GraphSubscription	Microsoft Graph webhook subscription lifecycle. provider_sub_id, client_state, expires_at.
Waitlist	Private-beta access control. status: pending/approved/rejected. source: signup/login.

Enrichment Repository

[backend/app/db/repository.py](#)

Data access layer for email_enrichment, audit_log, tone_profile, and processing_metric tables. All functions take a SQLAlchemy session as first argument.

Method / Function	Parameters	Description
upsert_enrichment	email_id, state, *, account_id, status, error	Insert or update enrichment row from pipeline state dict. Serialises axes, commitments, precedents to JSON.
get_enrichment	email_id, account_id → EmailEnrichment None	Fetch single enrichment row by email_id + account_id.
get_enrichments_bulk	email_ids: list, account_id → dict	Batch-fetch enrichments in one query (WHERE email_id IN ...). Returns dict keyed by email_id — critical for inbox page performance.
list_enrichments	*, account_id, priority, limit, offset → list	Paginated list with optional priority filter.
delete_enrichment	email_id, account_id → None	Hard delete (GDPR right-to-erasure).
write_audit	email_id, action, *, actor, details, account_id	Appends one audit log row. Never updates or deletes audit records.
record_priority_override	email_id, sender, override_priority, ...	Saves user priority correction. Also sets enrichment.status="done" AND mailbox_message.state="done" so email disappears from active inbox.
get_sender_priority_hint	sender, *, account_id → str None	Returns learned priority for a sender (from overrides). Used to bias triage scoring.
get_sender_priority_hints_bulk	senders: list, account_id → dict	Batch version of above for inbox page (one query for all senders).
get_audit_log	email_id, limit → list	Fetches audit trail for one email.
record_metric	email_id, stage, duration_ms, *, success, sla_met, account_id	Persists per-stage latency metric for SLA monitoring.
get_tone_profile	account_id → ToneProfile None	Fetch tone profile for an account.
save_tone_profile	account_id, profile: dict → None	Upsert tone profile (8 stylometric features).
purge_expired	retention_days: int → int	Deletes enrichments older than retention_days. Returns count of rows deleted.

Mailbox Repository

[backend/app/db/mailbox_repo.py](#)

Data access for the mailbox mirror: MailboxMessage and MailboxSyncState tables. Handles delta cursor persistence and state-preserving upserts.

Method / Function	Parameters	Description
-------------------	------------	-------------

get_sync_state	account_id, folder → MailboxSyncState None	Fetches delta cursor, backfill status, and last sync timestamp.
set_sync_status	account_id, folder, status, error → None	Updates last_status and last_error (for monitoring).
set_sync_cursor	account_id, folder, *, delta_cursor, backfill_done, status	Persists the new cursor after a successful delta sync.
recount	account_id, folder → int	Recomputes exact message_count from the DB (after upserts/tombstones).
upsert_messages	account_id, folder, messages: list → (new, updated)	Batch upserts envelope rows. Crucially: if row.state == "done", preserves it (does NOT reset to "active" — prevents done emails from reappearing after sync).
tombstone_messages	account_id, removed_ids: list → int	Sets state="deleted" for emails removed from the provider mailbox.
list_page	account_id, folder, limit, offset → list	LEFT OUTER JOIN mailbox_message + email_enrichment. Filters: state="active" AND (enrichment.status IS NULL OR enrichment.status != "done"). Returns envelope + triage data combined for the inbox page.

LangGraph State

[backend/app/graph/state.py](#)

Defines EmailAgentState TypedDict — the single shared state object flowing through all 6 pipeline nodes.

State Fields:

Input fields: email_id, sender, subject, body, received_at

PII masking (after ingest): masked_body, masked_subject, mask_mapping: dict[str, str]

Triage outputs: axes: list[AxisScore], dynamic_weights: dict[str, float], composite_score: float, priority: str, email_type: str, approval_mode: str, triage_reasoning: str

Commitment outputs: commitments: list[CommitmentItem], commitment_reasoning: str

Calendar outputs: conflict_summary: str, calendar_events: list[CalendarEvent]

RAG/Draft outputs: draft_reply: str, precedents: list[PrecedentItem]

Control fields: current_step: str, errors: list[str], approved: bool

LangGraph Pipeline Assembly

[backend/app/graph/pipeline.py](#)

Assembles the StateGraph from imported node functions. Defines edges: ingest → {trriage ■ commitment ■ rag} → calendar → gate. Exposes run_pipeline() for both sync and streaming execution.

Method / Function	Parameters	Description
build_mailmind_graph	index_documents: list → CompiledGraph	Creates StateGraph, registers 6 nodes, adds parallel edges, compiles. index_documents are passed into rag_node via closure for RAG retrieval.
run_pipeline	email_payload: dict, index_documents: list, stream: bool, parallel: bool → dict Generator	Main entry point. stream=True → returns SSE-compatible generator; stream=False → runs synchronously and returns full state dict. parallel=True uses asyncio.gather for triage+commitment+rag nodes.

Pipeline Nodes

[backend/app/agents/nodes.py](#)

The 6 agentic pipeline node functions and their helpers. Each node receives EmailAgentState and returns a partial state update dict. LLM calls use Azure OpenAI with Groq as fallback.

Method / Function	Parameters	Description
_get_llm	temperature: float, deployment: str → AzureChatOpenAI ChatGroq None	LRU-cached factory. Tries Azure OpenAI first, then Groq, then None. Cached so the SDK client is not re-instantiated per call.
_with_max_tokens	llm, max_tokens: int → BaseChatModel	Applies max_tokens limit in a provider-agnostic way (kwargs differ between Azure and Groq).
_dispatch_tool	tool_call: ToolCall → Any	Executes an LLM tool call returned by triage_node's tool-calling loop.
_priority_from_score	composite: float → str	Maps 0–100 composite score to CRITICAL / HIGH / MEDIUM / LOW string.
_normalise_weights	weights: dict → dict	Repairs LLM-returned dynamic_weights so they sum to exactly 1.0.

_is_automated_sender	sender: str, body: str → bool	Detects no-reply / automated sender patterns.
_dampen_automated_action	axes, sender, body → axes	Lowers action axis score for automated senders to prevent false HIGH priorities.
ingest_node	state: EmailAgentState → dict	Node 1: masks PII in subject + body with PIISanitizer, returns masked_body, masked_subject, mask_mapping.
triage_node	state: EmailAgentState → dict	Node 2: calls gpt-4o-mini with tool-calling schema for 5-axis scoring. Returns axes, dynamic_weights, composite_score, priority, email_type, approval_mode, triage_reasoning.
commitment_node	state: EmailAgentState → dict	Node 3: calls CommitmentService.extract() on masked_body. Returns commitments list with deadlines and confidence scores.
calendar_node	state: EmailAgentState → dict	Node 5: deterministic conflict detection — cross-references commitment deadlines against calendar events. Returns conflict_summary.
rag_node	state: EmailAgentState, index_documents: list → dict	Node 4: embeds masked_body with ada-002, searches ChromaDB (threshold 0.78), then generates draft via DraftService with RAG context injected. Returns draft_reply, precedents.
gate_node	state: EmailAgentState → dict	Node 6: approval checkpoint. In streaming mode is a no-op pass-through; in human-in-the-loop mode waits for /api/agent/approve/{email_id}. Sets approved=True.

Email Queue

[backend/app/queue/queue.py](#)

Queue abstraction layer. In development uses an in-memory deque; in production uses Redis BLPOP for durable, multi-worker job dispatch.

Method / Function	Parameters	Description
<code>EmailQueue.__init__</code>	<code>backend: str = settings.queue_backend</code>	Selects backend: "memory" → deque, "redis" → Redis BLPOP.
<code>EmailQueue.enqueue</code>	<code>job: dict → None</code>	Adds a job (email_id, account_id, provider) to the queue.
<code>EmailQueue.dequeue</code>	<code>→ dict None</code>	Pops next job (BLPOP blocks for 1s in Redis mode). Returns None when empty.
<code>EmailQueue.size</code>	<code>→ int</code>	Returns current queue depth.
<code>EmailQueue.clear</code>	<code>→ None</code>	Drains the queue (used in tests).

Enrichment Worker

[backend/app/workers/enrichment.py](#)

Background worker that dequeues enrichment jobs and runs the commitment+calendar+RAG nodes. Supports graceful shutdown via SIGTERM, exponential backoff retry, and SLA metrics.

Method / Function	Parameters	Description
<code>_load_rag_index</code>	<code>→ list</code>	Loads ChromaDB index documents from disk for rag_node.
<code>_restore_field</code>	<code>value: Any, mapping: dict → Any</code>	Restores PII tokens in a single field value.
<code>_restore_pii</code>	<code>state: dict → dict</code>	Restores all PII tokens in the full state dict after LLM calls.
<code>EnrichmentWorker.process_one</code>	<code>job: dict → dict</code>	Runs the enrichment subgraph (commitment → calendar → rag) for one email job. Reads existing triage state from DB, then runs Phase 2 nodes. Persists result.
<code>EnrichmentWorker.run</code>	<code>→ None</code>	Infinite loop: dequeue → process_one with exponential backoff retry (3 attempts, 30s base). Catches SIGTERM for graceful shutdown.

Observability

[backend/app/observability.py](#)

Global exception handlers, request ID middleware, and optional Sentry integration.

Method / Function	Parameters	Description
<code>init_observability</code>	<code>app: FastAPI → None</code>	Registers: unhandled exception handler (500 JSON response), request ID injection (X-Request-ID header), optional Sentry init if SENTRY_DSN is set.